

CS4432: Database Systems II

Query Processing Two-Pass Hash-Based Algorithms

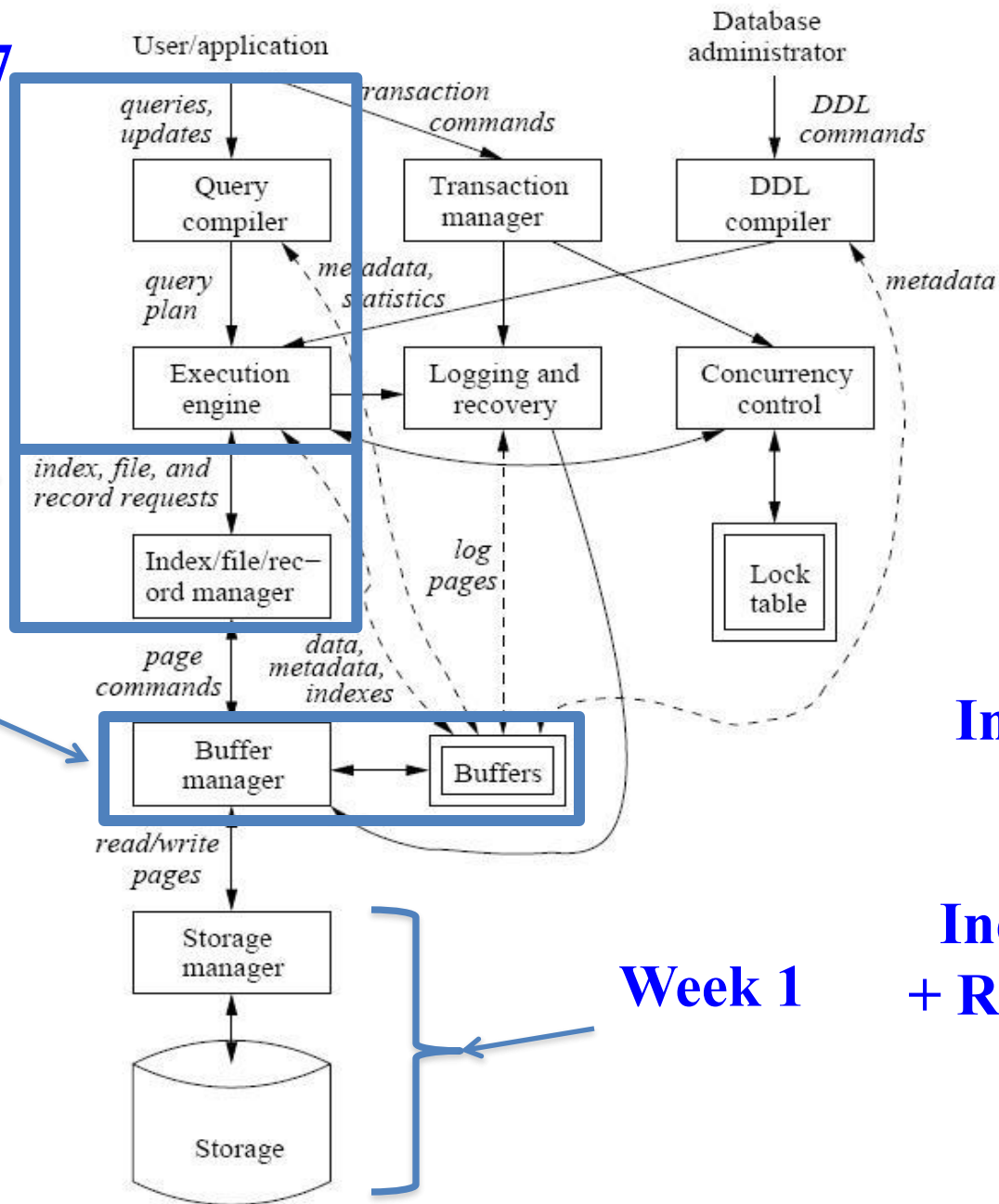
Updates

- **Final Exam This Thursday (April 30):**
 - You need to be in class to take it
 - **You are allowed to use a single-sided A4/letter with standard (1-inch) margins and at least a 12pt font size. The note can be handwritten or printed.**
- **HW 3 & Project 3:**
 - Due on May 4
 - No EXTENSIONS

Week 6 & 7

**Week 3:
sorting +**

Week 2



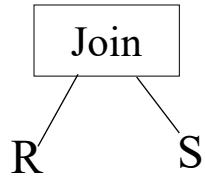
**Week 4:
Index via B+Tree**

**Week 5:
Index via Hashing
+ Relational Algebra**

Week 1

Covered So Far...

- **One-Pass Operator Evaluation**
 - Join, Duplicate Elimination, Group By, Set Union
- **Two-Pass Evaluation (Sort-Based)**
 - Sort, Duplicate Elimination, Join [Sort-Join, Sort-Merge-Join]



Example: Join (R,S)

One Pass

Iteration

Assume S is smaller than R
i.e., $B(S) < B(R)$

Open():

read S into memory

GetNext():

```
for b in blocks of R:
  for t in tuples of b:
    if t matches tuple s:
      return join (t,s)
return NotFound
```

Close():

Clean memory

- For this join algorithm to work:
 - S must fit in memory
 - One additional buffer for R

- **Key Metrics (memory Req.):**
 - $M \geq B(S) + 1$
- **I/O Cost:**
 - $B(S) + B(R)$

Distinct

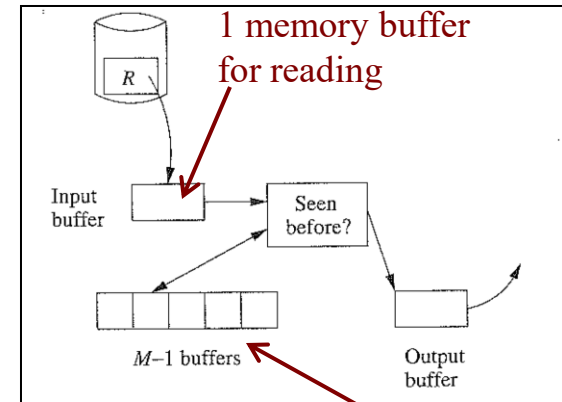
R

Example: Duplicate Elimination

One Pass

Iteration

- Keep a main memory search data structure D (use search tree or hash table) to store one copy of each tuple → **(M-1 Buffers)**
- Read in each block of R one at a time (use table scan) → **(1 buffer)**
- For each tuple check if it appears in D
 - If Yes, then skip
 - If Not, then add it to D and to the output buffer



M-1 memory buffers for storing distinct copies



What are the constraints for this algorithm to work in one pass?

The distinct tuples of R must fit in $M-1$ Buffers

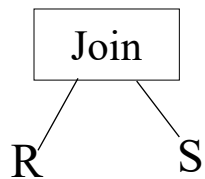
>> $B(\text{Distinct}(R)) \leq M-1$

>> As an approximation $B(\text{Distinct}(R)) \leq M$



What is the I/O Cost

$B(R)$



Efficient Two-Pass JOIN

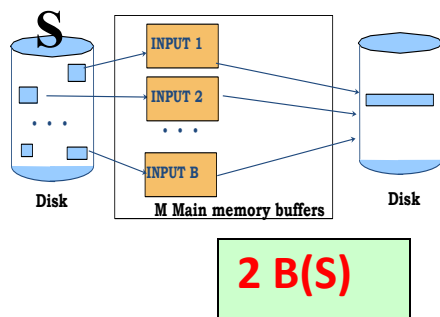
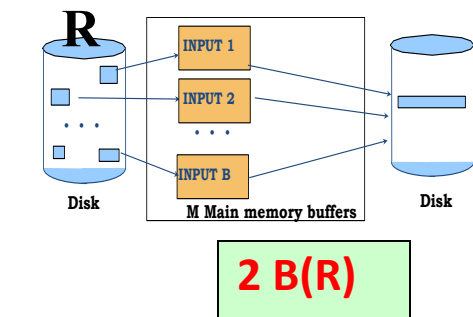
(*Sort-Merge-Join*)



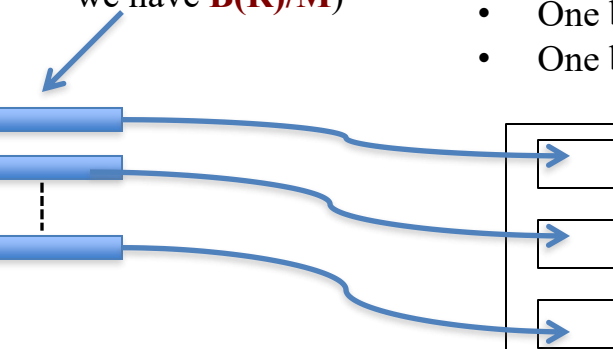
What is the I/O Cost

Main Idea: Combine Pass 2 of the Sort with the Join

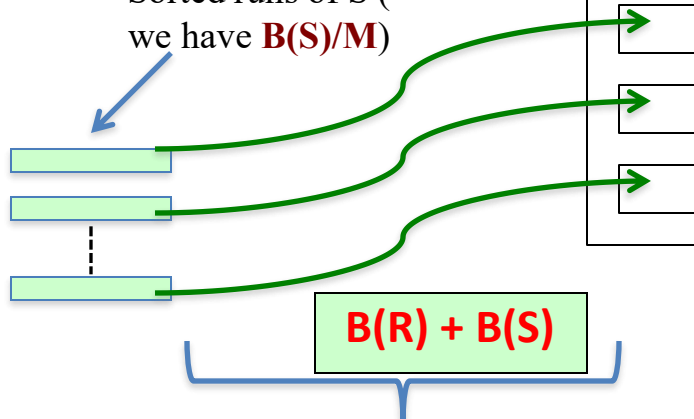
Phase 1 in Sorting As Is



Sorted runs of R (we have $B(R)/M$)

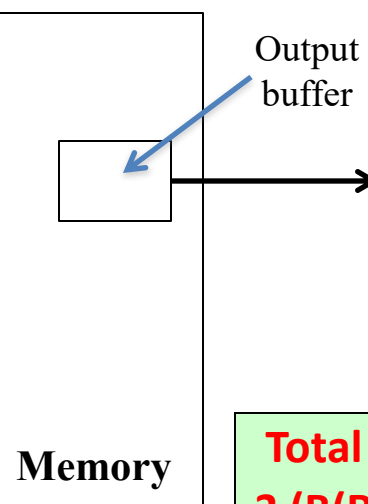


Sorted runs of S (we have $B(S)/M$)



Phase 2 Merge & Join

- One buffer for each sorted run from both R & S
- One buffer for the join output



Total Cost =
 $3 (B(R) + B(S))$

Covered So Far...

- **One-Pass Operator Evaluation**
 - Join, Duplicate Elimination, Group By, Set Union
- **Two-Pass Evaluation (Sort-Based)**
 - Sort, Duplicate Elimination, Join [Sort-Join, Sort-Merge-Join]

What's Next...

- **Two-Pass Evaluation (Hash-Based)**
 - Duplicate Elimination, Join

Learning Objectives

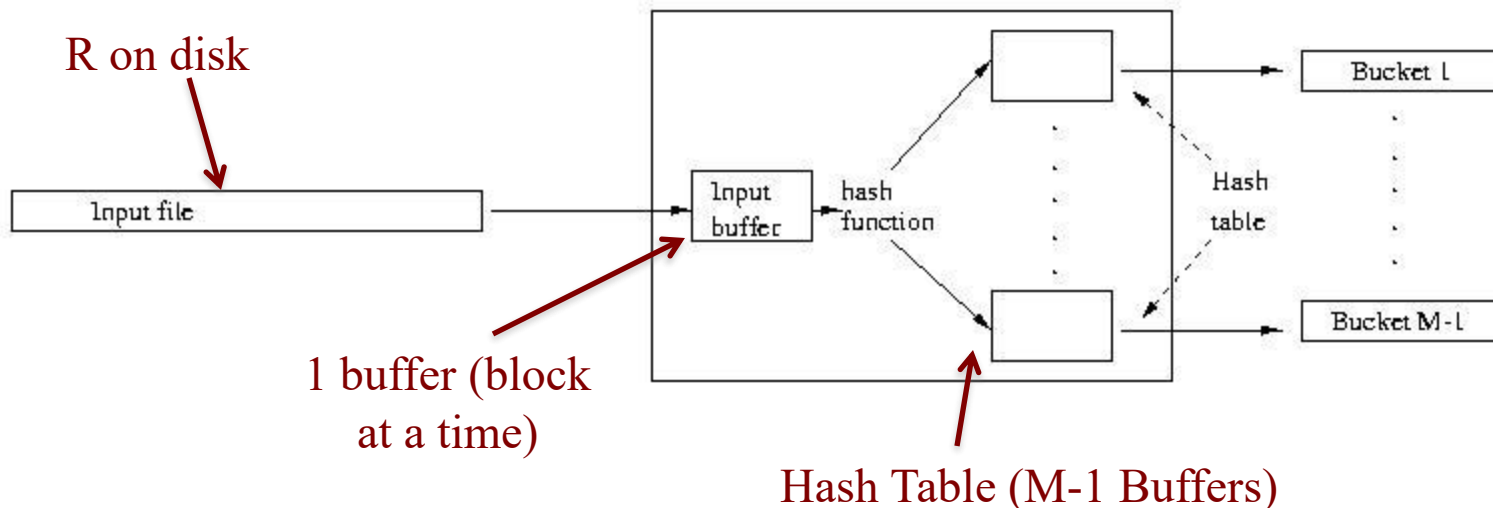
- Explain Hash-Based Two-Pass algorithms:
 - Duplicate Elimination
 - Join
- Derive pre-conditions and I/O cost for algorithms above
- Contrast hash-based and sort-based merge join

Hash-Based Algorithms: Main Idea

- Data is too large to fit in memory
- Partition the data (using hashing) into buckets
- Work on each bucket individually
 - Either One-Pass or Two-Pass

Partitioning a Relation

- Read one block at a time → (1 Buffer)
- Keep M-1 buffers for the hash table (M-1 buckets)
- Hash each tuple to its bucket's buffer
- If buffer of bucket X is full → write it to disk
- Good Hash function → Each bucket size = $B(R) / (M-1)$

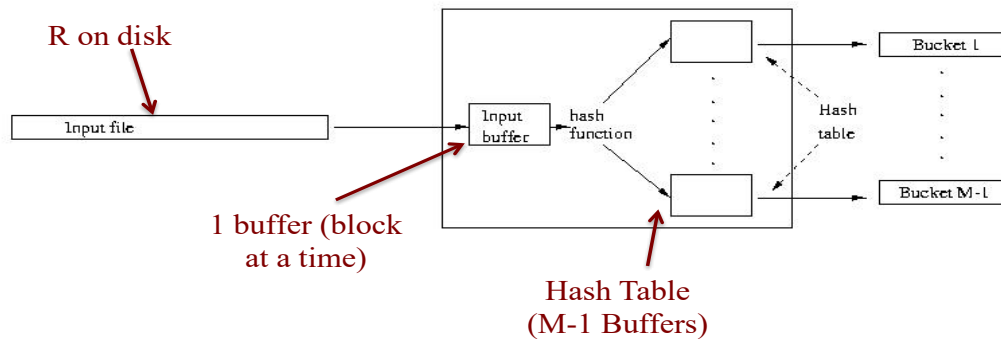


Distinct

R

Hash-Based Two-Pass Duplicate Elimination

- **Pass 1: Partition R using Hashing**



Each bucket size =
 $B(R) / (M-1)$

- **Pass 2: Load each bucket into memory and eliminate duplicates**
 - Identical tuples must exist in the same bucket



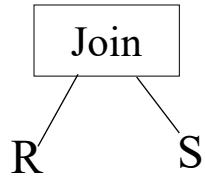
What are the constraints for this algorithm to work in two pass?

- A bucket must fit in memory
- $B(R)/M-1 \leq M \rightarrow B(R) < M^2$



What is the I/O Cost

- Pass 1 $\rightarrow 2 B(R)$, Pass 2 $\rightarrow B(R)$
- **Total $\rightarrow 3 B(R)$**

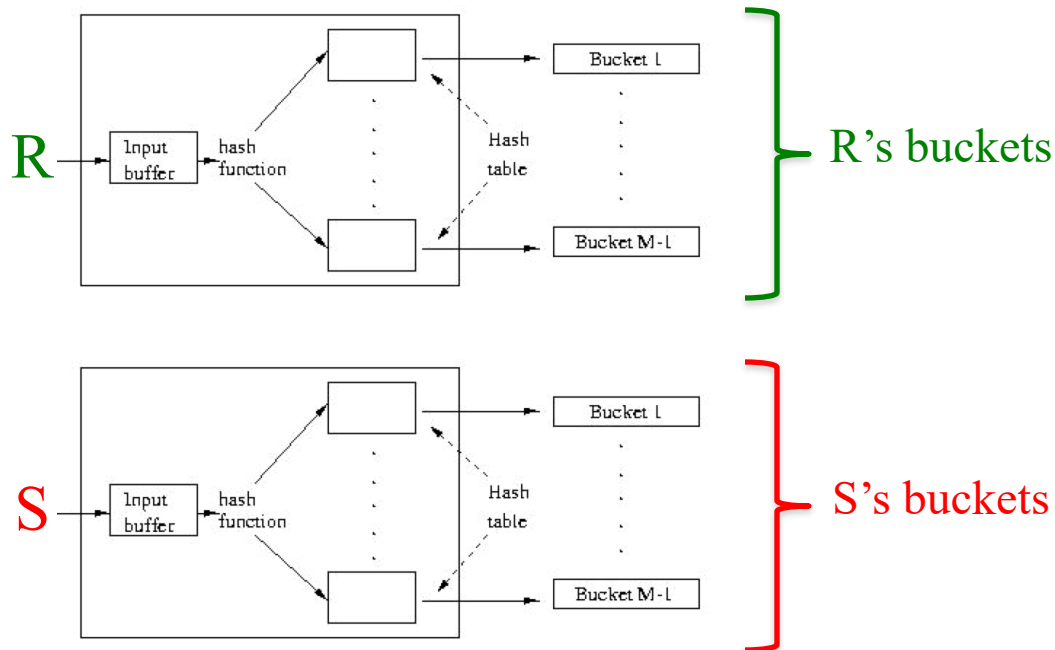


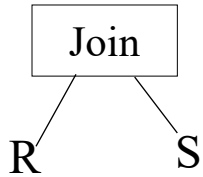
Hash-Based Two-Pass Join

$$R.x = S.y$$

- **Phase 1:** Partition each relation using the same hash function into buckets
 - Hash function must be on join key
- **Phase 2:** Join Bucket i from R with Bucket i from S

• Phase 1: Partition

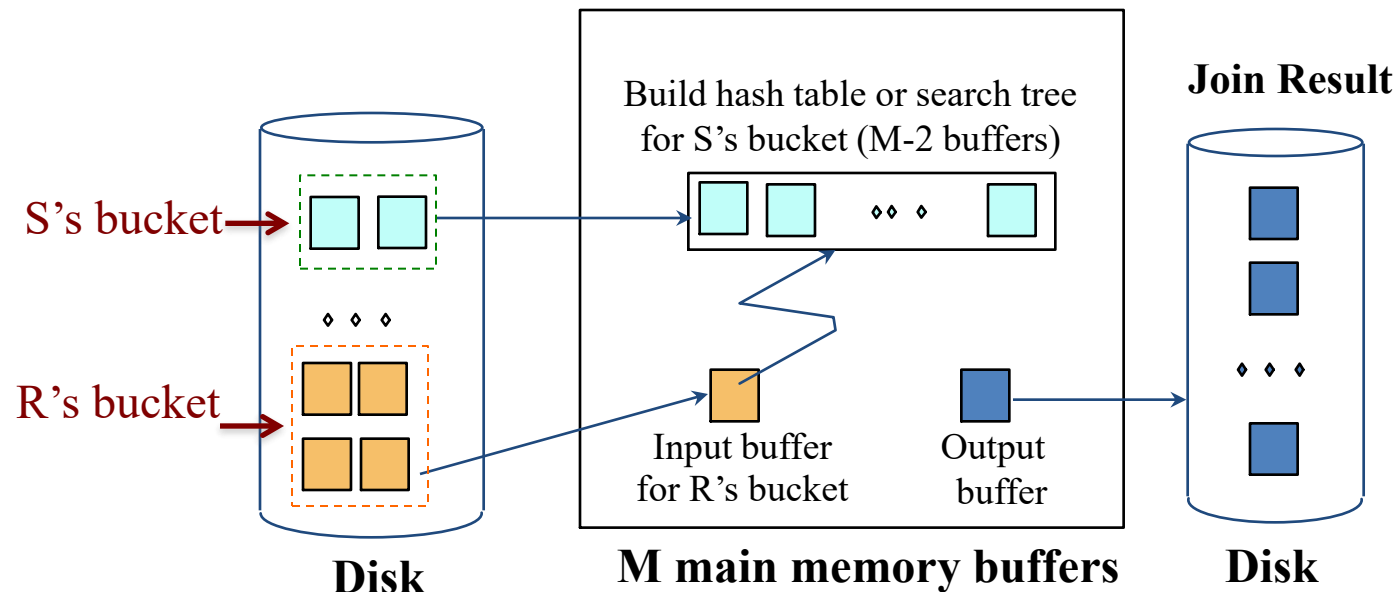




Hash-Based Two-Pass Join (Cont'd)

- Phase 2: Join Buckets R.x and S.y**

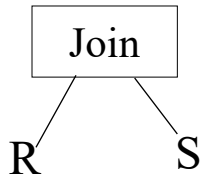
- Move the smaller bucket to memory → (M-2) buffers
- Read from the other bucket one block at a time and join → 1 buffer



What are the constraints for this algorithm to work in two pass?



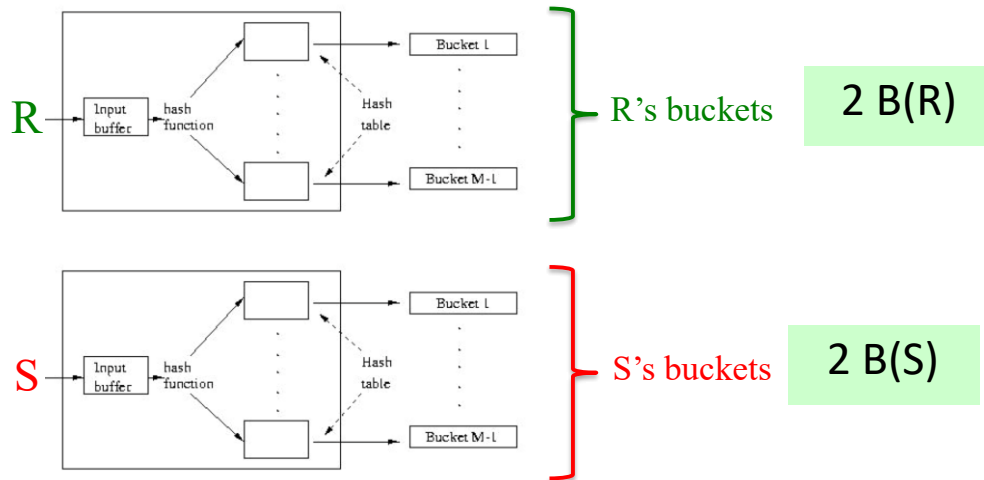
What is the I/O Cost



Hash-Based Two-Pass Join

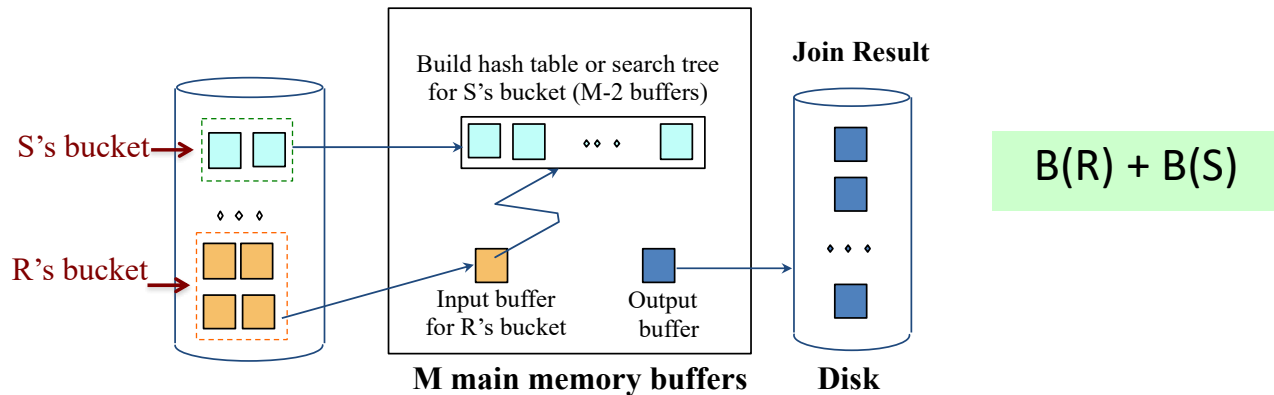
What is the I/O Cost

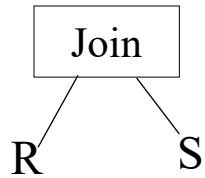
• Phase 1: Partition



Total I/O cost =
 $3(B(R) + B(S))$

• Phase 2: Join Buckets R.x and S.y

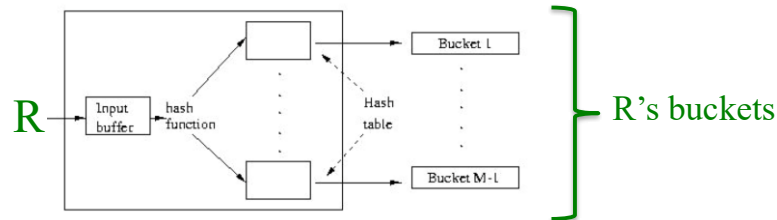




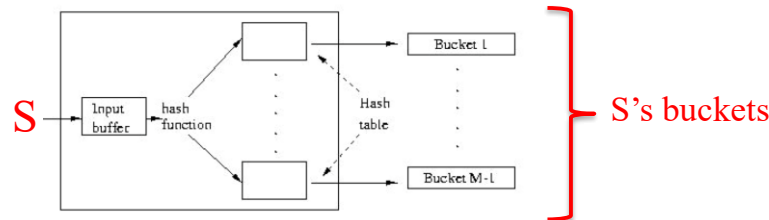
Hash-Based Two-Pass Join

What are the constraints?

• Phase 1: Partition

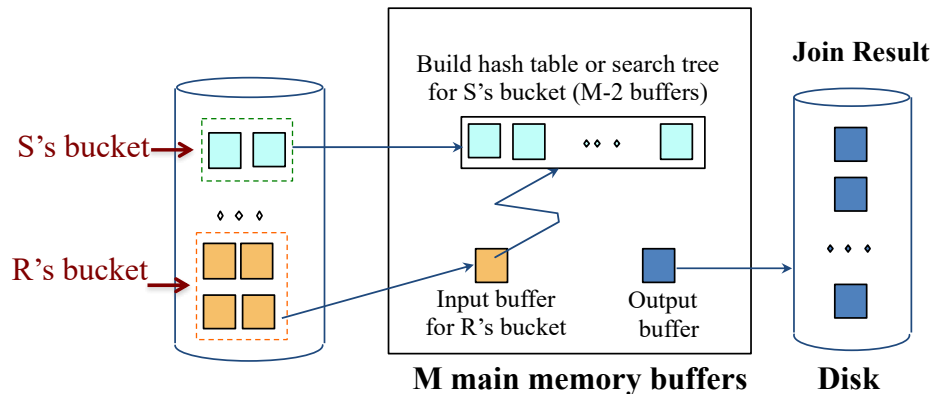


No new constraints



No new constraints

• Phase 2: Join Buckets R.x and S.y



Smaller bucket must fit in memory

→ $B(S)/M \leq M$ (approximation)

→ $B(S) \leq M^2$

Or $\text{Min}(B(R), B(S)) \leq M^2$

Summary

- **One-Pass Operator Evaluation**
 - Join, Duplicate Elimination, Group By, Set Union
- **Two-Pass Evaluation (Sort-Based)**
 - Sort, Duplicate Elimination, Join [Sort-Join, Sort-Merge-Join]
- **Two-Pass Evaluation (Hash-Based)**
 - Duplicate Elimination, Join
- **Multi-Pass algorithms?**
 - Same ideas, more passes 😊

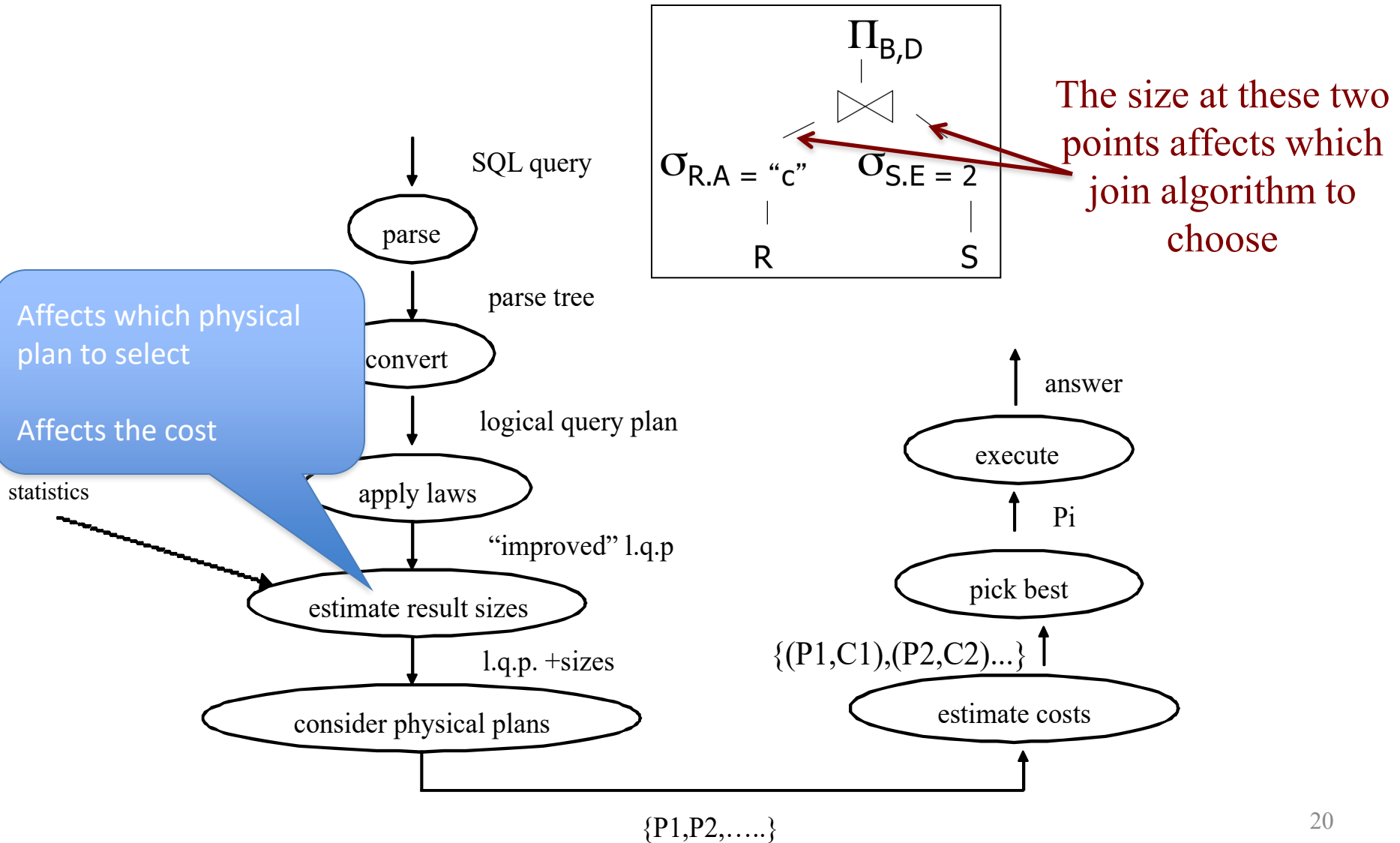
CS4432: Database Systems II

Size Estimation

Learning objectives

- Identify the basic statistics over a relation needed by the DBMS and their notation
- Estimate size of operators:
 - Selection: equality, range, combination
 - Join and sequence of joins

Overview of Query Execution



Common Statistics over Relation R

- $B(R)$: # of blocks to hold all R tuples
- $T(R)$: # tuples in R
- $S(R)$: # of bytes in each of R 's tuple
- $V(R, A)$: # distinct values in attribute $R.A$

We care about computing these statistics for each intermediate relation

Requirements for Estimation Rules

1. Give accurate estimates
2. Are easy (fast) to compute
3. Are logically consistent: estimated size should not depend on *how the relation* is computed

Here we describe some simple heuristics.

Estimating Size of Selection

$$R' = \sigma_p(R)$$

- **Equality Condition: $R.A = c$** , where c is a constant
 - Reasonable estimate $T(R') = T(R) / V(R,A)$
 - i.e.: Original # of tuples divided by # of unique values of A
- **Range Condition: $c1 < R.A < c2$:**
 - If $R.A$ domain is known $D \rightarrow T(R') = T(R) \times (c2 - c1) / D$
 - Otherwise $\rightarrow T(R') = T(R) / 3$
- **Non-Equality Condition: $R.A \neq c$**
 - A good estimate $\rightarrow T(R') = T(R)$

- **$B(R)$** : # of blocks to hold all R tuples
- **$T(R)$** : # tuples in R
- **$S(R)$** : # of bytes in each of R 's tuple
- **$V(R, A)$** : # distinct values in attribute A

Example

- Consider relation $R(a,b,c)$ with 10,000 tuples and 50 different values for attribute a .
- Consider selecting all tuples from R with $(a = 10 \text{ and } b < 20)$.
- Estimate of number of resulting tuples:
 - $10,000 * (1/50) * (1/3) = 67$.

If condition is the AND of several predicates → estimate in series.

Estimating Size of Selection (Cont'd)

If condition has the form C_1 **OR** C_2 , use:

1. **Sum** of estimate for C_1 and estimate for C_2 , Or

2. Assuming C_1 and C_2 are independent,

$$T(R) * (1 - (1-f_1)*(1-f_2)),$$

where f_1 is fraction of R satisfying C_1 and

f_2 is fraction of R satisfying C_2

- $R(a,b) \rightarrow$ 10,000 tuples and 50 different values for a .
- Estimate
 - Estimate for $a = 10$ is $10,000/50 = 200$
 - Estimate for $b < 20$ is $10,000/3 = 3333$
 - Estimate for combined condition is
 - $200 + 3333 = 3533$, OR
 - $10,000 * (1 - (1 - 1/50) * (1 - 1/3)) = 3466$
 - Different, but not much

Select from R with
($a = 10$ or $b < 20$)

Estimating Size of Natural Join

$$U = R \bowtie S$$

- Assume join is on a single attribute Y.
- Some Possibilities:
 1. R and S have disjoint sets of Y values, so size of join is 0
 2. Y is the key of S and a foreign key of R, so size of join is $T(R)$
 3. All the tuples of R and S have the same Y value, so size of join is $T(R)*T(S)$
- We need some assumptions...

Expected number of tuples in result is:

$$T(U) = T(R)*T(S) / \max(V(R,Y), V(S,Y))$$

Rationale: Size of Natural Join

- If Cartesian Product, then size is $T(R) \times T(S)$
- In natural join, in addition to uniform assumption, we make the “subset assumption”:
 $\text{distinct}(R.Y) \subseteq \text{distinct}(S.Y)$ or $\text{distinct}(S.Y) \subseteq \text{distinct}(R.Y)$,
- Example: $T(R) = T(S) = 1,000$, $V(R,Y) = 10$ and $V(S,Y) = 20$
- Given a random pair $(r.y, s.y)$, what is the prob that $r.y = s.y$?
 - Answer: $1/V(S,Y) = 0.05$
- $T(U) = 1,000 * 1,000 / 20 = 50,000$

Expected # of tuples in result is:
 $T(U) = T(R) * T(S) / \max(V(R,Y), V(S,Y))$

Example:

$$Z = R1(A,B) \bowtie R2(B,C) \bowtie R3(C,D)$$

R1	$T(R1) = 1000$	$V(R1,A)=50$	$V(R1,B)=100$
----	----------------	--------------	---------------

R2	$T(R2) = 2000$	$V(R2,B)=200$	$V(R2,C)=300$
----	----------------	---------------	---------------

R3	$T(R3) = 3000$	$V(R3,C)=90$	$V(R3,D)=500$
----	----------------	--------------	---------------

Partial Result: $U = R1 \bowtie R2$

$$Z = R1(A,B) \bowtie R2(B,C) \bowtie R3(C,D)$$

R1	$T(R1) = 1000$	$V(R1,A)=50$	$V(R1,B)=100$
----	----------------	--------------	---------------

R2	$T(R2) = 2000$	$V(R2,B)=200$	$V(R2,C)=300$
----	----------------	---------------	---------------

R3	$T(R3) = 3000$	$V(R3,C)=90$	$V(R3,D)=500$
----	----------------	--------------	---------------

$$T(U) = \frac{1000 \times 2000}{200}$$

$$V(U,A) = 50$$

$$V(U,B) = 100$$

$$V(U,C) = 300$$

$$Z = U \bowtie R3$$

$$T(Z) = \frac{1000 \times 2000 \times 3000}{200 \times 300}$$

$$V(Z,A) = 50$$

$$V(Z,B) = 100$$

$$V(Z,C) = 90$$

$$V(Z,D) = 500$$

$$Z = R1(A,B) \bowtie R2(B,C) \bowtie R3(C,D)$$

R1	T(R1) = 1000	V(R1,A)=50	V(R1,B)=100
----	--------------	------------	-------------

R2	T(R2) = 2000	V(R2,B)=200	V(R2,C)=300
----	--------------	-------------	-------------

R3	T(R3) = 3000	V(R3,C)=90	V(R3,D)=500
----	--------------	------------	-------------

T(U)	1000	2000	3000
------	-----------------	-----------------	-----------------

3000	200	?	??
-----------------	-----	---	----

V(U,A)	50	?
--------	---------------	---

V(U,B)	100	?
--------	----------------	---

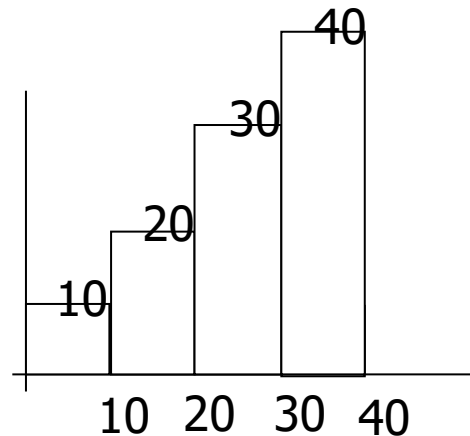
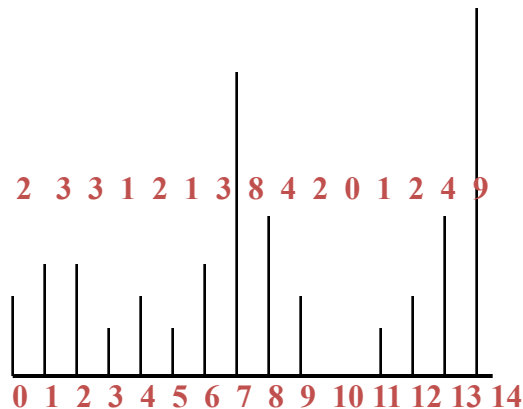
V(U,C)	300	?
--------	----------------	---

Summary of Estimation Rules

- Projection: exactly computable
- Product: exactly computable
- Selection: reasonable heuristics
- Join: reasonable heuristics
- The other operators are harder to estimate...

More on Estimation

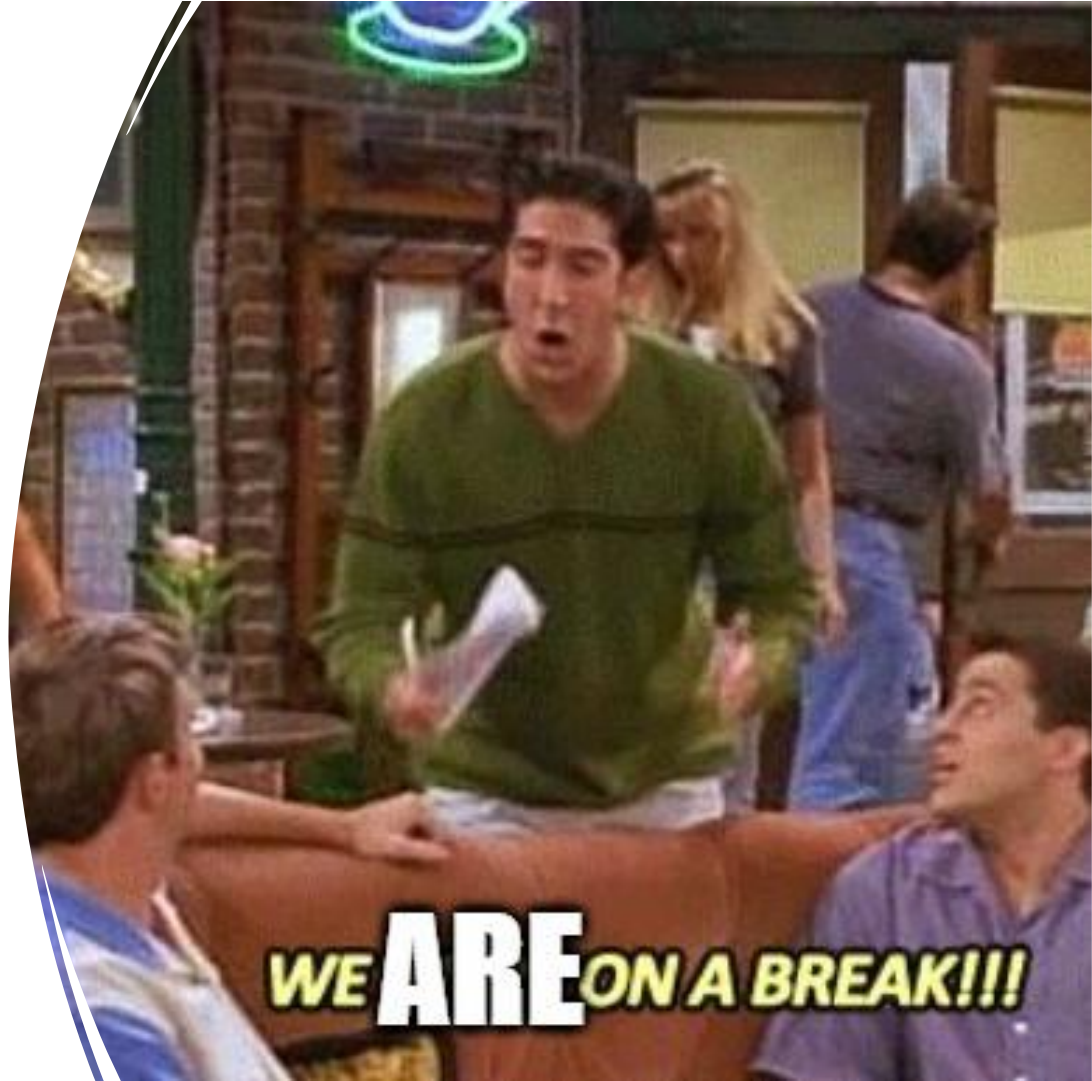
- Uniform distribution is not accurate since real data is not uniformly distributed.
- **Histogram:**
 - A data structure maintained by a DBMS to approximate a data distribution.
 - Divide range of column values into subranges (buckets).
Assume distribution within histogram bucket is uniform.



number of tuples
in R with A value
in given range

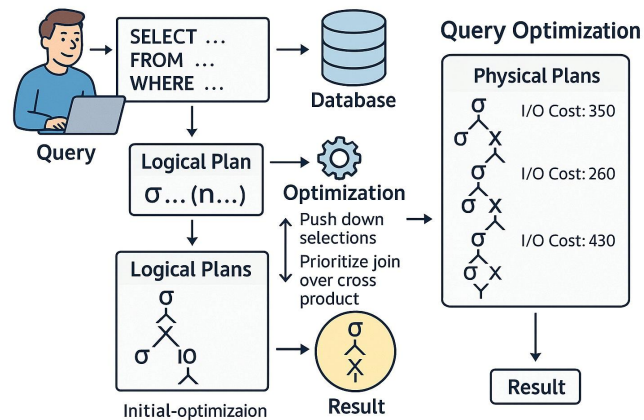
Break

- Now we will take a few minutes break



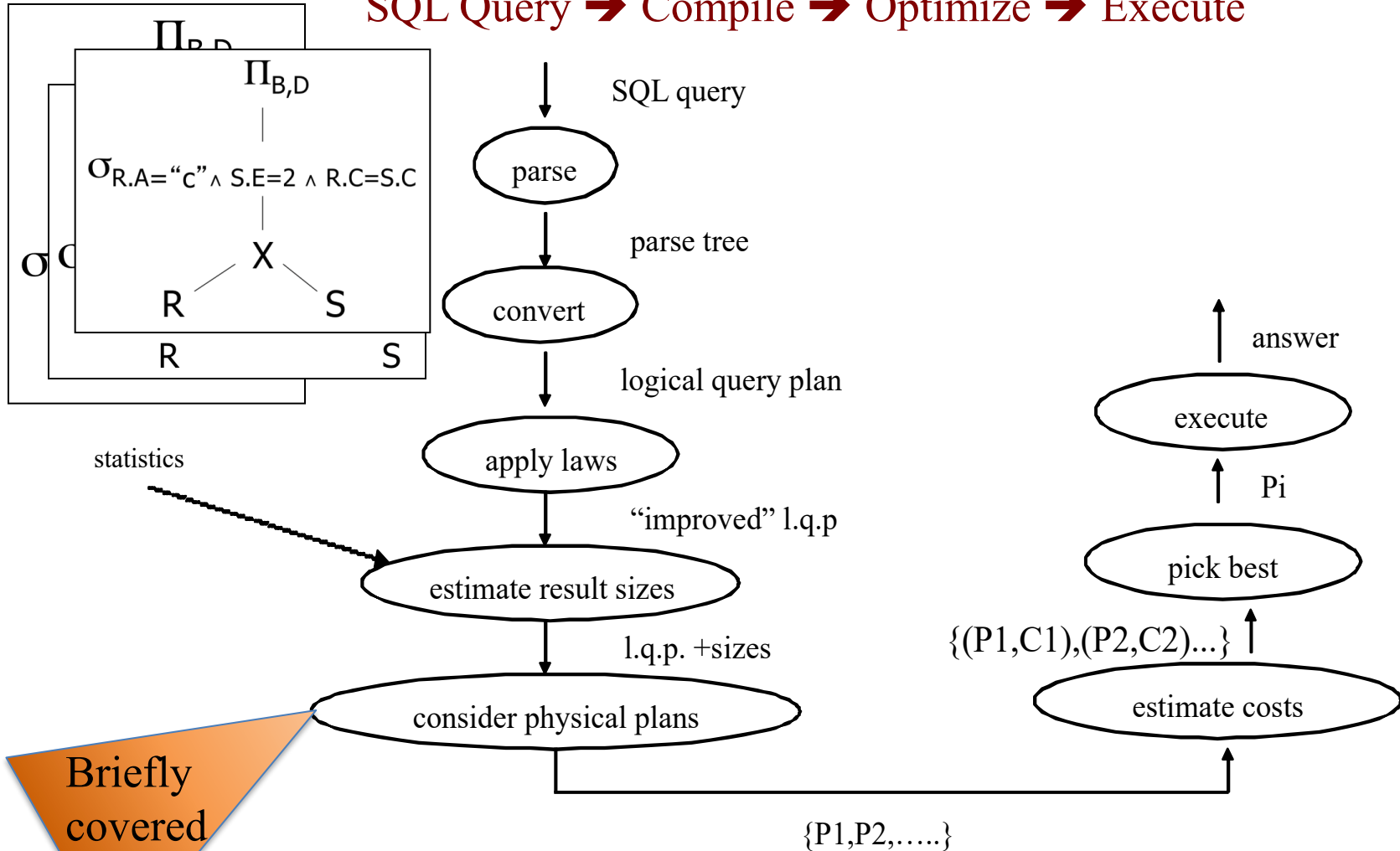
CS4432: Database Systems II

Query Rewrite



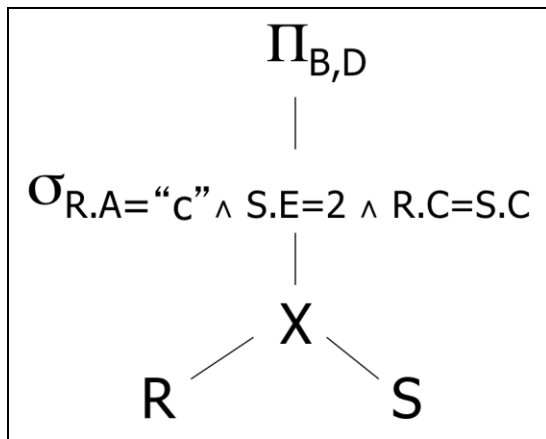
Overview of Query Execution

SQL Query $\rightarrow$ Compile $\rightarrow$ Optimize $\rightarrow$ Execute

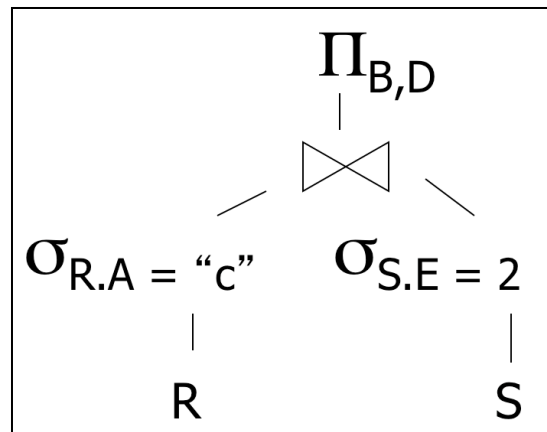


Example

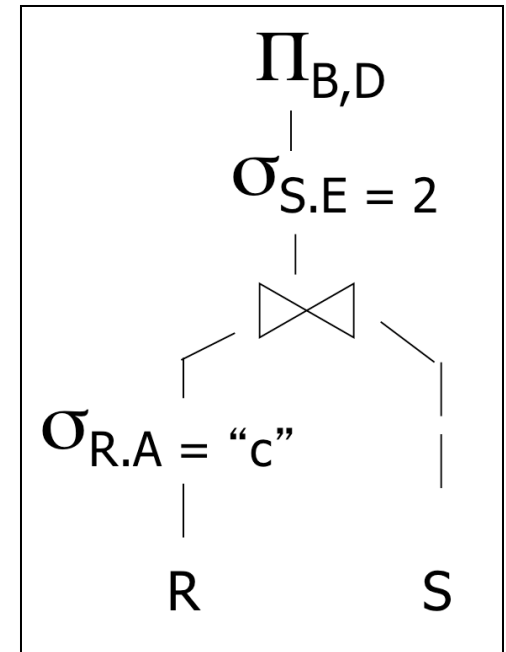
Select B, D **From** R, S **Where** R.A = "c" **And** S.E = 2 **And** R.C = S.C



Plan 1



Plan 2



Plan 3

How are these plans generated?

Relational Algebra Optimization

- Set of rules to apply called “**Transformation rules**” or “**Algebraic Laws**”
- **What are transformation rules ?**
 - Preserve equivalence of plans
 - That is, must produce the same answer
- **What are good transformations?**
 - Reduce query execution costs

Relational Operators (revisited)

- Selection Basics

- Idempotent $\sigma_A(R) = \sigma_A \sigma_A(R)$

- Commutative $\sigma_A \sigma_B(R) = \sigma_B \sigma_A(R)$

- Selection Conjunctions

- Useful when pruning $\sigma_{A \wedge B}(R) = \sigma_A(\sigma_B(R)) = \sigma_B(\sigma_A(R))$

- Selection Disjunctions

- Equivalent to UNIONS $\sigma_{A \vee B}(R) = \sigma_A(R) \cup \sigma_B(R)$

Rules: Selection and Binary Operators

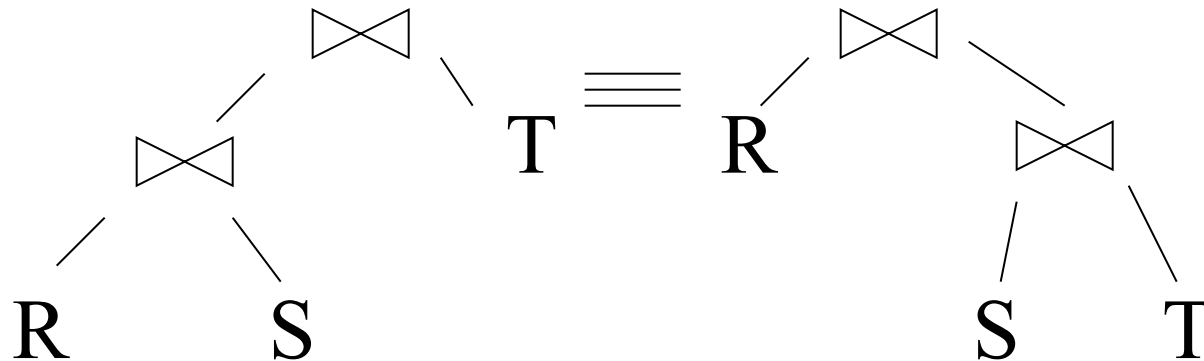
- Must push selection to both arguments:
 - $\sigma_C(R \cup S) = \sigma_C(R) \cup \sigma_C(S)$
- Must push to first arg, optional for 2nd:
 - $\sigma_C(R - S) = \sigma_C(R) - S$
 - $\sigma_C(R - S) = \sigma_C(R) - \sigma_C(S)$
- Push to at least one arg with all attributes mentioned in C:
 - product, natural join, theta join, intersection
 - e.g., $\sigma_C(R \times S) = \sigma_C(R) \times S$, if R has all the attributes in C

Rules: Natural Join Rewriting

$$R \bowtie S = S \bowtie R$$

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

Can also write as trees, e.g.:



Rules: $\sigma + \bowtie$ Combined

Let p = predicate with only R attributes

q = predicate with only S attributes

m = predicate with both R and S attributes

$$\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$$

$$\sigma_q(R \bowtie S) = R \bowtie \sigma_q(S)$$

$$\sigma_m(R \bowtie S) = \text{No change}$$



Join Predicates

Always a good idea to
push selection down

Rules: $\sigma + \bowtie$ Combined

Let p = predicate with only R attributes

q = predicate with only S attributes

m = predicate with both R and S attributes

What about these ones??

$$\sigma_{p \wedge q \wedge m} (R \bowtie S) = \sigma_m [(\sigma_p R) \bowtie (\sigma_q S)]$$

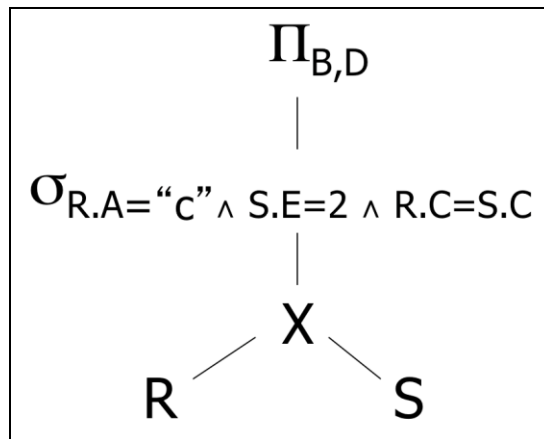
$$\sigma_{p \vee q} (R \bowtie S) = [(\sigma_p R) \bowtie S] \cup [R \bowtie (\sigma_q S)]$$

What should we know/do?

- Rewrite rules transform a query into an **equivalent** one that is **cheaper** to execute, without changing the result.
- Always Good: **Selection Push-Down; Join over cross product**
- Sometimes Good: **Projection Push-Down**
- **Other than that:** Compare cost of logical plans (Should I R S or S R) and physical plans (Should I use sort-based or hash-based join)

An Improved Logical Query Plan

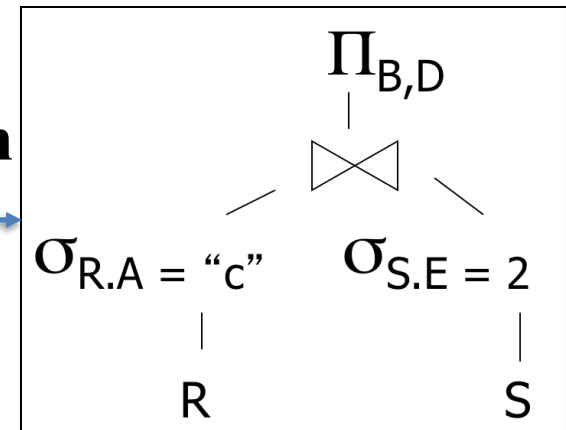
Select B, D **From** R, S **Where** R.A = "c" **And** S.E = 2 **And** R.C = S.C



Plan 1

Selection Push-Down

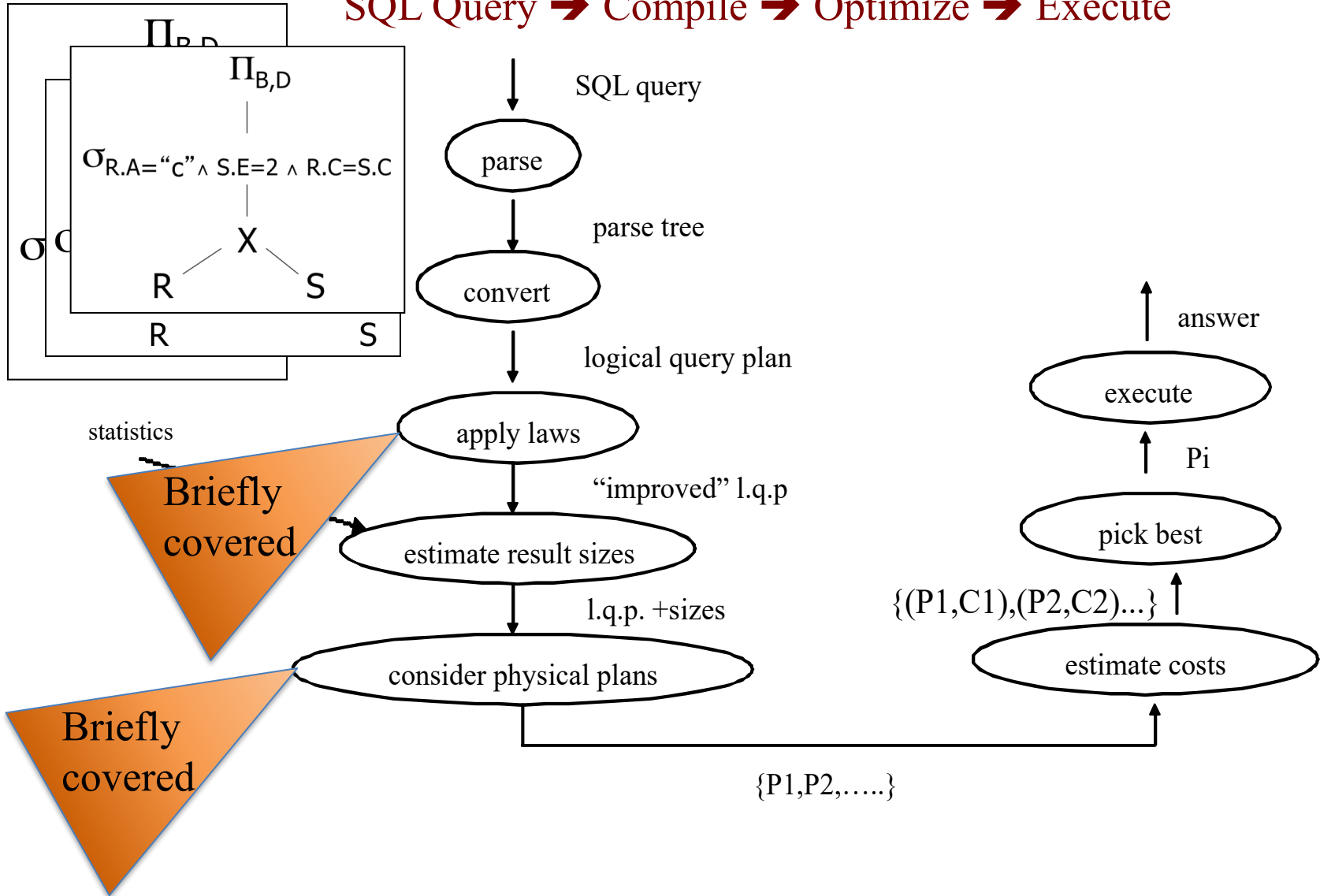
**Join over
cross product**



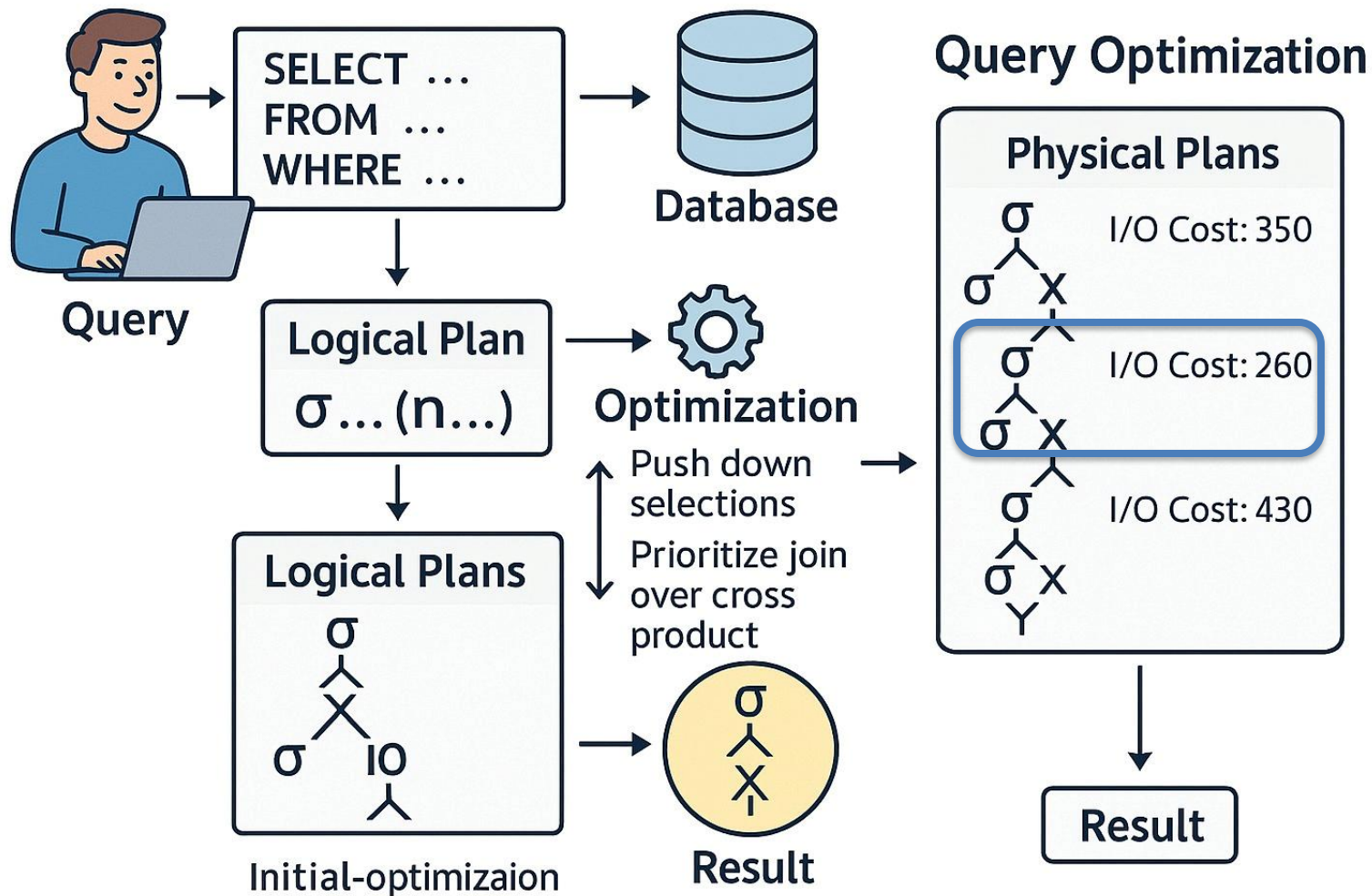
Plan 2

Overview of Query Execution

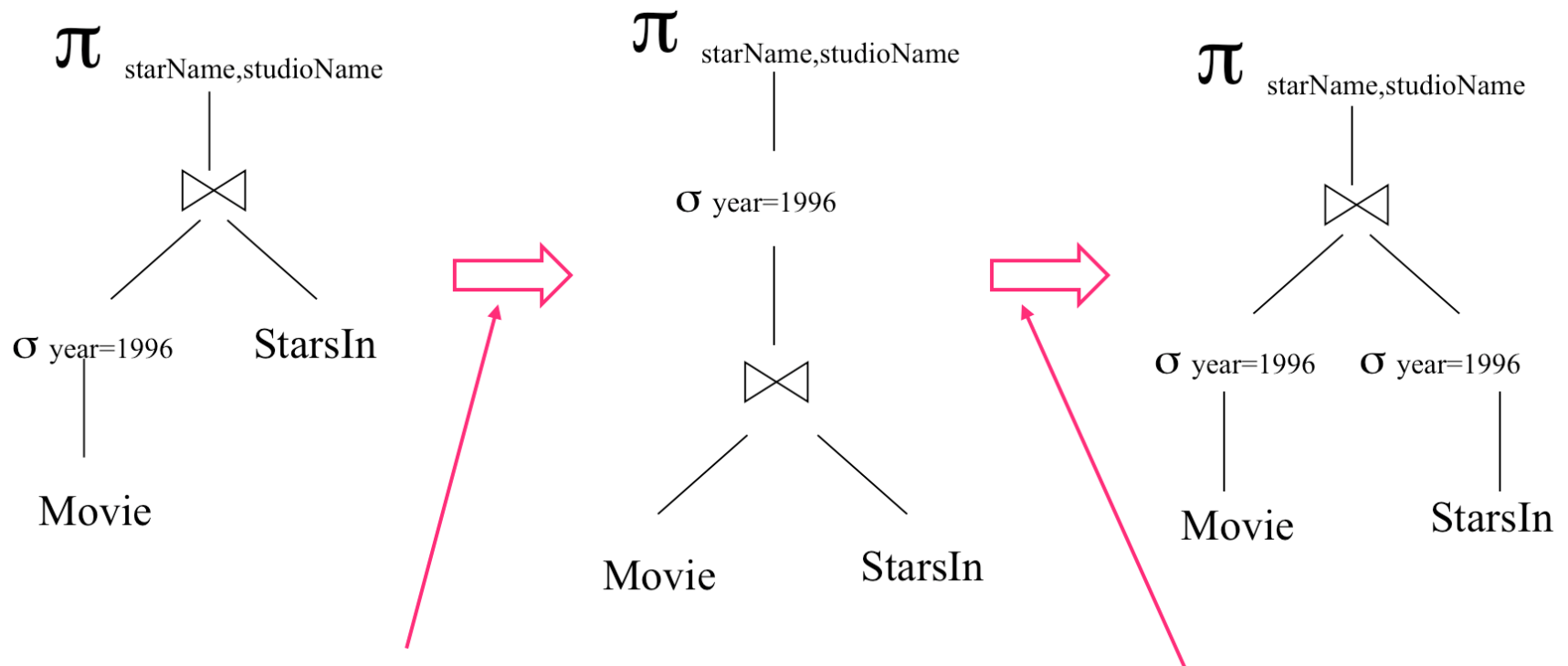
SQL Query $\rightarrow$ Compile $\rightarrow$ Optimize $\rightarrow$ Execute



Bigger Picture



An (edge-case) Logical Query Plan



push selection
up tree

$$\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$$

push selection
down tree (both sides)

$$\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$$

$$\sigma_q(R \bowtie S) = R \bowtie \sigma_q(S)$$

Summary of Query Rewrite

- Transformation rules to create equivalent query plans
 - Check textbook for more rules
 - Always select-push-down is good
 - Sometimes project-push-down is good
- } Both reduce the size as early as possible
- Pushing selection all the way → enables using indexes
 - Order among join relations $R \bowtie S \rightarrow S \bowtie R$
 - Affects which one is *outer* or *inner*